ASSIGNMENT

Submission Timeline: 48 Hours

Objective

Build a deployed, agentic application that accepts multiple input types simultaneously (Text, Images, PDFs, Audio files), extracts content, understands the user's goal, and autonomously performs the correct task — including complex, multi-step queries that require chaining several tools in a single request.

If the query or goal is not clear, the agent must ask a follow-up question before acting.
All final outputs must be text-only.

1. Inputs Supported

The agent must support all of the following input types, and must be able to accept multiple types in a single request:

- Text (plain or structured)
- Image (JPG/PNG) → OCR
- PDF (text or scanned) → PDF parsing + OCR fallback
- Audio (MP3/WAV/M4A) → Speech-to-Text + cleanup
- Multiple inputs simultaneously — e.g., a PDF + a text query together

2. Agent Behaviour

A. Intent Understanding

The agent must:

- Extract or transcribe content from all provided inputs
- Identify the user's goal across the combined context
- Detect constraints (timing, format, instructions)
- Resolve cross-input references — e.g., a URL found inside a PDF should be fetched when the user's query implies it

B. Mandatory Follow-Up Question Rule

If the input does not contain enough information to determine the task, or if multiple tasks are equally plausible, the agent must not guess. It must ask a short, clear follow-up question such as:

- "Could you clarify whether you want a summary or sentiment analysis?"
- "What do you want me to do with this extracted text?"

- "Should I explain this code or rewrite it?"

The agent should ONLY proceed after receiving clarity.

The agent must plan the minimum viable tool sequence and execute it without requiring step-by-step user prompting.

3. Tasks the Agent Must Handle

1. Image/PDF Text Extraction — Return cleaned transcript + OCR confidence.
2. YouTube Transcript Fetching — Detect URL anywhere in any input → fetch transcript (or fallback message).
3. Conversational Answering — Friendly, helpful response for general questions.
4. Summarization — Output must include: 1-line summary, 3 bullets, 5-sentence summary.
5. Sentiment Analysis — Label + confidence + one-line justification.
6. Code Explanation — Explain what code does, detect bugs, and mention time complexity.
7. Audio Transcription + Summary — Convert audio → text → summarize (same 3 formats).
8. Cross-Input Reasoning — Combine content from multiple inputs to answer a unified query.

4. Deployment Requirement

The application must be deployed and accessible via a public URL. Local-only submissions will not be accepted.

Accepted deployment targets include:

- Render / AWS / GCP / Azure
- Any cloud platform capable of running a FastAPI backend
- Docker-based deployments are encouraged for reproducibility

The submission must include:

- The live deployment URL
- A Dockerfile or deployment configuration file
- Instructions for environment variable setup (API keys, etc.)

5. UI Requirements

- One text input box for queries
- File upload supporting Image, PDF, and Audio — multiple files selectable at once
- Clean, minimal chat-like UI (chat UI is strongly preferred)

- Text-only output panel
- Show extracted text alongside the final agent result
- Display the agent's reasoning / tool chain steps taken (plan trace)

6. Deliverables

- Clean, modular codebase
- Architecture diagram (showing input pipeline, agent core, tool registry, output)
- FastAPI backend + simple UI frontend
- Dockerfile / deployment config
- Live deployment URL
- Test cases (see Section 8)
- README with setup, usage, and design decisions

7. Evaluation Rubric (100 Points)

Criterion	Description
Correctness (30)	Tasks produce correct outputs across all input types and multi-input combinations
Autonomy & Planning (20)	Agent plans sensible, minimal-step workflows; handles multi-tool chaining autonomously
Robustness (15)	Error handling, retries, partial results, graceful degradation on bad inputs
Explainability (10)	Readable plan + logs returned for each run; multi-step traces visible in UI
Code Quality & Modularity (10)	Clean structure, linting, DI, tests, deployment config included
UX & Demo (10)	Usable UI, clear demo walkthrough, sample inputs covering complex query examples

Minimum passing score: 75 / 100

8. Sample Test Cases

Test Case 1 — Audio Transcription + Summary

Input: Audio lecture (5 min)

Expected: Agent transcribes → produces 1-line summary + 3 bullets + 5-sentence summary + duration.

Test Case 2 — PDF + Natural Language Query

Input: PDF (3 pages) containing meeting notes + query: "What are the action items?"
Expected: Agent extracts text → finds and returns action items only.

Test Case 3 — Image with Code

Input: Image screenshot containing a code snippet + prompt "Explain"
Expected: Agent OCRs → detects language → explains + warns about any bugs.

Test Case 4 — Cross-Input Multi-Tool Chain (New)

Input: PDF containing a YouTube URL + query: "Hit the YT URL in this PDF and give me a summary of it"
Expected: Agent extracts PDF text → detects YouTube URL → fetches transcript → returns 1-line + bullets + 5-sentence summary. No user prompting between steps.

Test Case 5 — Multi-File Unified Query (New)

Input: An audio file + a PDF document + query: "Do the audio and the document discuss the same topic?"
Expected: Agent transcribes audio → parses PDF → compares content → returns a comparative analysis.

9. Bonus (Extra Credit)

- Cost estimator: approximate token/API cost per plan prior to execution, shown in the UI.
- Streaming output: stream agent responses token-by-token in the chat UI.
- Tool call visualization: show a real-time graph or step list of which tools were invoked and in what order.